WISE ROUND 3: COMPLETE SYSTEM DESIGN GUIDE

Mobile Engineering Manager — Spend Squad

60 min | Two Engineers | Zoom

PART 1: WHAT WISE OFFICIALLY TESTS

From Wise's own System Design Interview page (wise.jobs):

> "We will present a **product task** or problematic area and we expect you to propose a **high-level architecture** approach. Additionally, we may ask for further details and delve into **lower-level aspects** such as defining data structures, or clarifying specific logic."

The 4 Assessment Criteria:

Criteria	What They're Looking For	How to Show It
Product thinking	"Think from the customer's perspective"	Start with user journey, not database schema
Problem solving	"Approach to solving a non-trivial problem"	Structured breakdown, don't jump to solution
Communication	"Justify your decisions, openness to feedback"	"I chose X over Y because..."
Technical proficiency	"Relevant to Wise's technical stack"	VIPER, TCA, SwiftUI, Combine, Kafka

The Round Structure (from Wise + Glassdoor):

```
0:00 - 0:05   Introductions — they introduce themselves
0:05 - 0:15   Experience discussion — your past projects
0:15 - 0:20   They present the design problem
0:20 - 0:25   You clarify requirements
0:25 - 0:40   You design the architecture (high-level → detailed)
0:40 - 0:50   They add requirements / probe deeper
0:50 - 0:55   Trade-offs, testing, production discussion
0:55 - 1:00   Your questions for them
```

What Got People REJECTED (Glassdoor/Blind):

> "Design was easy but expected to cover **more depth** which I couldn't in given time" — Blind "Should experience be synchronous or async with notification? When I heard that I wanted to say **depends**... but I swallowed it" — Glassdoor Nov 2025 "Pattern was contention and scaling reads/writes with financial twist. **I didn't justify trade-offs** and got confused" — Reddit

The #1 rejection reason: Not going deep enough + not justifying trade-offs.

PART 2: EXPERIENCE DISCUSSION (First 10 min)

From Wise's page:

> "Expect some questions about your **past experiences and projects**. Your **contributions**, decisions you have taken, **recent bugs you've fixed**."

Prepare These 3 Stories (2 min each):

Story 1: "Tell me about a technical decision you made"

> "On BFX at PayPal, I advocated for country-by-country rollout over simultaneous launch for re-enabling currency conversion on Visa multi-currency cards. Product wanted speed, I pushed for safety. We ramped Italy and Saudi Arabia first, caught a BIN configuration issue in Italy that would have affected all other markets. Fixed it before expanding. The phased approach protected customers while hitting the $15M revenue target. The decision was to trade launch speed for customer safety — and it paid off because we avoided a multi-country incident."

Story 2: "Tell me about a bug you recently fixed"

> "RUNE Send/Request page displayed incorrectly after fresh login — buttons appeared that shouldn't be visible, and the title was missing. Worked fine after app relaunch. I traced it to the experiment flag `Trmt_p2p_money_movement_flow` — the `qualifiedTreatment` was nil during fresh login because the experiment assignments were fetched asynchronously and hadn't completed before the UI rendered. Cached values from the previous session were cleared on logout. Fix: load cached experiment values synchronously on startup, then refresh from server in background. Documented the pattern as a debugging playbook for the team."

Story 3: "Tell me about a system you designed"

> "I created the complete BFX knowledge transfer system — not just documentation, but a reproducible process. Step-by-step deployment checklists, troubleshooting guides with the 10 most common issues, access request templates, and a 4-week onboarding plan. Reduced new engineer onboarding from 4 weeks of shadowing to 1 week of self-guided ramp-up. The document became the team's living reference — versioned, with a quarterly review cycle."

PART 3: THE DESIGN FRAMEWORK

Step 1: Clarify Requirements (2-3 min)

When they present the problem, ask:

Functional:

> "What's the core user journey? What does the customer see step by step?"
> "Should this work offline? What happens with no network?"
> "Are there different flows for different countries/currencies?"

Non-functional:

> "What's the latency requirement? Sub-second?"
> "How many concurrent users? Thousands? Millions?"
> "What's the consistency requirement — eventual or strong?"

Scope:

> "Should I focus on the mobile architecture, or include backend as well?"
> "Should I design for iOS only, or consider both platforms?"
> "Should I start with the happy path and then discuss error handling?"

Step 2: Start with User Journey (2 min)

ALWAYS start from the customer, not from the database.

> "Let me start with what the customer experiences:
> Step 1: Customer taps their Wise card at a POS terminal
> Step 2: They get an instant notification — merchant name, amount, currency
> Step 3: They open the app and see the transaction in their list
> Step 4: They can tap for details, dispute if needed
> Now let me design the architecture that makes this happen..."

Why this matters: Wise's #1 criterion is "product thinking." Starting from the customer's perspective shows you're a product engineer, not just a systems architect.

Step 3: Draw High-Level Architecture (5 min)

Draw these layers:

```
┌──────────────────────────────────────────────────────────────┐
│ EXTERNAL                                                     │
│ Card Networks (Visa/Mastercard) → APNs → Merchant Systems    │
└──────────────────────────────────────────────────────────────┘
                │
┌──────────────────────────────────────────────────────────────┐
│ WISE BACKEND                                                 │
│                ▼                                             │
│ API Gateway → Payment Processor → Event Bus (Kafka)          │
│                        │              │                      │
│            ┌───────────┴──────────────┴──────────┐           │
│            ▼            ▼                        │           │
│        Notification    Transaction              │           │
│        Service         Service                  │           │
│        (→ APNs)        (→ PostgreSQL)           │           │
└──────────────────────────────────────────────────────────────┘
                │
┌──────────────────────────────────────────────────────────────┐
│ MOBILE APP (VIPER)                                           │
│                ▼                                             │
│ Push Handler → Router → Presenter → Interactor → Repository  │
│                              │    │    │                     │
│                              APIClient Cache                 │
└──────────────────────────────────────────────────────────────┘
```

What to SAY while drawing:

> "I'm drawing three layers — external systems that Wise integrates with, Wise's backend services, and the mobile app. Events flow from the card network through Wise's backend to the mobile app via push notifications. Let me zoom into the mobile architecture..."

Step 4: Zoom Into Mobile — VIPER Architecture (10 min)

This is where you shine. You know VIPER.

For each feature, draw the VIPER module:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                                                          │
│  BUILDER (assembles the module)                                          │
│    Creates all components, wires dependencies                            │
│                                                                          │
│  VIEW (Protocol) ←→ PRESENTER ←→ INTERACTOR                              │
│  │                  │           │              │                        │
│  │ Shows UI         │           │ Business logic │                      │
│  │ Sends user actions │         │ Fetches data   │                      │
│  │ Completely dumb    │         │ Validates      │                      │
│  │                  │           │              │                        │
│  │                ROUTER      ENTITY            │                        │
│  │                  │           │              │                        │
│  │               Navigates   Pure data          │                        │
│  │               to screens  models             │                        │
│  │                                              │                        │
│  │        REPOSITORY (called by Interactor)     │                        │
│  │        ├── APIClient (network)               │                        │
│  │        └── LocalCache (storage)              │                        │
│  │                                              │                        │
└──────────────────────────────────────────────────────────────────────────┘
```

The 5 things to say about VIPER:

1. **View:** "Completely dumb — only renders what Presenter says. No business logic, no formatting, no network calls."
2. **Presenter:** "The coordinator. Receives user actions from View, asks Interactor for data, formats results for display, tells Router to navigate. It's the only component that talks to both View and Interactor."
3. **Interactor:** "Pure business logic — like a Use Case in Clean Architecture. Validates input, coordinates data fetching, applies business rules. Doesn't know about UI."
4. **Router:** "Owns all navigation. When the user taps a transaction, Presenter tells Router, and Router pushes the detail screen. Navigation logic is isolated — changing screen flow doesn't touch business logic."
5. **Entity:** "Pure data models. No dependencies, no frameworks. Just the data structures — Transaction, SpendControls, DisputeReason."

Step 5: Deep Dive on the Hard Part (10 min)

Every feature has ONE hard part. Identify it and go deep:

Feature	The Hard Part
Card transactions	Real-time push + multi-currency display
3DS authentication	Time-limited push → approve → result within 5 min
Card ordering	Physical vs virtual + Apple Wallet integration
Bill split	Rounding + Wise vs non-Wise users
Recurring payments	Detection algorithm + merchant blocking
Disputes	File upload + regulatory timelines

Step 6: When They Add Requirements (10 min)

They'll say something like: "*Now how would you add spending categories?*" or "*What if we need offline support?*"

Template:

> "Good addition. I'd approach this by:
>
> 1. **New component, not modify existing** — I'd create a new Interactor/Presenter for the spending summary, not bloat the transaction list module
> 2. **Feature flag** — wrap it behind `spending_categories_enabled` for gradual rollout
> 3. **Backend-driven** — categories assigned by backend per transaction. Mobile just displays them
> 4. **Trade-off:** client-side categorization is faster but inconsistent across devices. Server-side is consistent but requires backend work. I'd choose server-side because financial data must be consistent."

Step 7: Close with Production Concerns (5 min)

Always volunteer these at the end:

```
TESTING:
  "Each VIPER component is independently testable.
   Mock the View to test Presenter.
   Mock the Repository to test Interactor.
   No real network, no real UI — tests run in milliseconds."

MONITORING:
  "I'd track: notification delivery latency (< 1 sec),
   transaction list load time (< 300ms),
   crash-free rate (> 99.9%),
   API error rate per endpoint."

FEATURE FLAGS:
  "Roll out behind a flag. 5% of users first, monitor for a week,
   then ramp. Exactly how I manage BFX rollouts at PayPal."

SECURITY:
  "Certificate pinning for all API calls.
   Sensitive data in Keychain, not UserDefaults.
   Biometric auth for high-value actions (3DS approval, card freeze).
   Screen protection when app enters background (blur balance)."

ACCESSIBILITY:
  "VoiceOver labels on all transaction amounts and merchant names.
   Dynamic Type support. Minimum 44x44 tap targets."
```

PART 4: THE 6 SPEND SQUAD SCENARIOS

SCENARIO 1: CARD TRANSACTION + NOTIFICATION (★★★★★ Most Likely)

Prompt: "When a customer taps their Wise card, they get an instant notification. Design the mobile architecture."

User Journey:

```
Card tap → POS → Visa → Wise backend → Push → Mobile shows notification
                                     → Transaction in list
                                     → Balance updated
```

VIPER Module:

```swift
```

```swift
// ═══ ENTITY ═══
struct CardTransaction: Codable, Equatable {
    let id: String
    let merchantName: String
    let merchantCategory: String
    let amount: Decimal          // What merchant charged
    let currency: String         // Currency merchant charged in
    let convertedAmount: Decimal? // What was deducted from balance
    let balanceCurrency: String?  // Balance currency
    let rate: Decimal?           // FX rate if multi-currency
    let status: TransactionStatus
    let timestamp: Date
}

// ═══ INTERACTOR ═══
class TransactionListInteractor: TransactionListInteractorProtocol {
    weak var output: TransactionListInteractorOutput?
    private let repository: TransactionRepositoryProtocol
    private let pushHandler: PushHandlerProtocol

    func fetchTransactions() {
        // 1. Show cached data immediately
        if let cached = repository.getCached() {
            output?.didFetchTransactions(cached)
        }

        // 2. Refresh from network in background
        Task {
            do {
                let fresh = try await repository.fetchFromServer()
                await MainActor.run {
                    output?.didFetchTransactions(fresh)
                }
            } catch {
                await MainActor.run {
                    output?.didFailFetching(error)
                }
            }
        }
    }

    func handlePushNotification(_ data: [String: Any]) {
        // 3. Parse push → insert new transaction at top
        guard let txn = parsePushData(data) else { return }
        repository.insertLocal(txn)
        output?.didReceiveNewTransaction(txn)
```

```
    }
}

// ═══ PRESENTER ═══
class TransactionListPresenter {
    weak var view: TransactionListViewProtocol?
    var interactor: TransactionListInteractorProtocol
    var router: TransactionListRouterProtocol
    private let formatter = CurrencyFormatter()

    // Format for display:
    // "€45.00 at Starbucks Paris"
    // "S$65.25 from SGD balance · Rate: 1 EUR = 1.45 SGD"
    func format(_ txn: CardTransaction) -> TransactionCellViewModel {
        TransactionCellViewModel(
            merchantName: txn.merchantName,
            primaryAmount: formatter.format(txn.amount, currency: txn.currency),
            secondaryAmount: txn.convertedAmount.map {
                formatter.format($0, currency: txn.balanceCurrency ?? "")
            },
            rateInfo: txn.rate.map { "Rate: 1 \(txn.currency) = \($0) \(txn.balanceCurrency ?? "")" },
            statusIcon: iconFor(txn.status),
            timeAgo: RelativeDateFormatter().format(txn.timestamp)
        )
    }
}

// ═══ ROUTER ═══
class TransactionListRouter: TransactionListRouterProtocol {
    func navigateToDetail(_ transaction: CardTransaction) {
        let detailModule = TransactionDetailBuilder.build(transaction)
        // Push detail screen
    }

    func navigateToDispute(_ transaction: CardTransaction) {
        let disputeModule = DisputeFlowBuilder.build(transaction)
        // Present dispute flow modally
    }
}
```

Multi-Currency Display (the hard part):

> "When a customer pays €45 in Paris but their balance is SGD, we show both amounts — the charged amount and the deducted amount. This is exactly what I work on at PayPal with BFX — the rate determination, fee calculation, and dual-currency display. The key is always showing the customer's money in their balance currency as the primary amount, with the original charge currency secondary."

Push Notification Architecture:

```
Background: UNNotificationServiceExtension
  → Download merchant logo
  → Format rich notification
  → "💳 Starbucks — €45.00 (S$65.25 from SGD)"

Foreground: NotificationCenter observer
  → Show in-app banner with haptic
  → Auto-dismiss after 5 seconds
  → Tap → navigate to transaction detail

Both: Insert transaction into local list (optimistic)
  → Refresh from API to confirm
  → Update balance display
```

Trade-offs to discuss:

> **Push vs Polling:** "Push for real-time events, polling for periodic refreshes. Card transactions need sub-second notification, so push is the only option. But I'd add a pull-to-refresh fallback in case push fails."
> **Optimistic insert:** "I show the transaction from push data immediately, then confirm with API. The trade-off is the push might have preliminary data — amount could change (restaurant tips). So I'd show 'Pending' status and update when settled."

SCENARIO 2: 3DS AUTHENTICATION (★★★★)

Prompt: "Online purchases sometimes require 3DS verification. Design the mobile 3DS flow."

User Journey:

```
User's card used online → Wise receives 3DS challenge →
Push notification → User opens app → "Approve S$150 at Amazon?" →
User approves with Face ID → Result sent to Wise → Transaction approved
```

VIPER Module:

```swift
```

```swift
// ═══ ENTITY ═══
struct ThreeDSChallenge: Codable {
    let transactionReference: String
    let merchantName: String
    let amount: Decimal
    let currency: String
    let expiresAt: Date      // Must respond before this time
}

enum ChallengeResult: String {
    case approved = "APPROVED"
    case declined = "DECLINED"
}

// ═══ INTERACTOR ═══
class ThreeDSInteractor: ThreeDSInteractorProtocol {
    weak var output: ThreeDSInteractorOutput?
    private let apiClient: SpendAPIClientProtocol
    private let biometricAuth: BiometricAuthProtocol

    func approve(challenge: ThreeDSChallenge) {
        Task {
            // Step 1: Require biometric authentication
            guard try await biometricAuth.authenticate(
                reason: "Approve \(challenge.currency) \(challenge.amount) at \(challenge.merchantName)"
            ) else {
                await MainActor.run { output?.didFailAuth("Biometric cancelled") }
                return
            }

            // Step 2: Check not expired
            guard challenge.expiresAt > Date() else {
                await MainActor.run { output?.didExpire() }
                return
            }

            // Step 3: Submit to Wise API
            // POST /v3/spend/profiles/{id}/3dsecure/challenge-result
            do {
                try await apiClient.submit3DSResult(
                    reference: challenge.transactionReference,
                    status: .approved
                )
                await MainActor.run { output?.didSubmitResult(.approved) }
            } catch {
                await MainActor.run { output?.didFailSubmission(error) }
```

```
            }
        }
    }
}

// ═══ PRESENTER ═══
class ThreeDSPresenter {
    private var countdownTimer: Timer?

    func startCountdown(expiresAt: Date) {
        countdownTimer = Timer.scheduledTimer(withTimeInterval: 1, repeats: true) {
            [weak self] _ in
            let remaining = Int(expiresAt.timeIntervalSinceNow)
            if remaining <= 0 {
                self?.countdownTimer?.invalidate()
                self?.view?.showExpired()
            } else {
                self?.view?.updateCountdown(remaining)
                if remaining < 60 {
                    self?.view?.showUrgentCountdown() // Red color
                }
            }
        }
    }
}
```

Key concerns to mention:

> **Timing:** "5-minute expiry window from Wise's API. Push must arrive within seconds. Countdown timer in the UI with red warning under 60 seconds."
> **Deep linking:** "Push notification must open the 3DS approval screen directly — even if app was killed. I'd use a custom URL scheme in the push payload."
> **Biometric auth:** "Before approving, require Face ID / Touch ID. Stolen phone shouldn't be able to approve transactions."
> **Idempotency:** "Wise's API processes only the first call — duplicates are ignored. Safe for network retries and double-taps."
> **Fallback:** "If push doesn't arrive (phone offline), Wise falls back to SMS OTP. Mobile should handle both flows."

SCENARIO 3: CARD ORDERING FLOW (★★★★)

Prompt: "Design the card ordering experience."

VIPER with Coordinator (multi-step flow):

```
CardOrderCoordinator (manages the flow)
├── VIPER Module 1: CardTypeSelection
│   └── Physical / Virtual, Debit / Prepaid
│
├── VIPER Module 2: DeliveryAddress (physical only)
│   └── Dynamic Forms — backend sends field structure per country
│   └── Address validation API
│
├── VIPER Module 3: ReviewOrder
│   └── Summary, fees, estimated delivery
│   └── Biometric auth before confirming
│
├── VIPER Module 4: OrderConfirmation
│   └── Virtual: instant → show card details + Add to Apple Wallet
│   └── Physical: tracking status → push on each stage
│
└── Each step's Router delegates to the Coordinator for flow navigation
```

Key points:

> **Dynamic Forms:** "Wise uses server-driven UI. The backend sends form structure per country — different countries require different address fields. Mobile renders whatever the backend sends, no app update needed."
> **Virtual vs Physical:** "Virtual cards are instant — I'd show the card details immediately and offer PassKit integration to add to Apple Wallet. Physical cards need a tracking flow with push notifications at each stage: ordered → printed → shipped → delivered."
> **Idempotency:** "Card ordering must be idempotent. If the user taps Confirm twice or the network retries, they shouldn't get two cards. I'd include an idempotency key in the request header."

SCENARIO 4: BILL SPLIT (★★★)

Prompt: "After a restaurant payment, the user can split the bill with friends."

Architecture:

```swift
```

```
// Interactor handles the split logic
class BillSplitInteractor {
    func splitEqually(amount: Decimal, participants: Int) -> [Decimal] {
        let share = amount / Decimal(participants)
        let rounded = share.rounded(2)  // Round to 2 decimal places
        let total = rounded * Decimal(participants - 1)
        let lastShare = amount - total  // Last person absorbs rounding

        var shares = Array(repeating: rounded, count: participants - 1)
        shares.append(lastShare)
        return shares
    }
    // S$100 ÷ 3 = [S$33.33, S$33.33, S$33.34]
    // Total is still exactly S$100 — no money created or lost
}
```

Key discussion:

> **Rounding:** "Fintech rule — never create or lose money through rounding. Last person absorbs the remainder. S$100 ÷ 3 = S$33.33 + S$33.33 + S$33.34."
> **Wise-to-Wise vs external:** "If the friend has Wise, instant free request. If not, send a payment link via SMS or WhatsApp. Two very different flows — the Router decides which to present."

SCENARIO 5: DISPUTE FLOW (★★★)

Prompt: "A customer wants to dispute a transaction."

Multi-step VIPER with Coordinator:

```
DisputeCoordinator
├── Step 1: SelectReason (enum of predefined reasons)
├── Step 2: ProvideDetails (text + photo evidence upload)
├── Step 3: ReviewAndSubmit (summary + biometric confirm)
└── Step 4: TrackResolution (status timeline + push updates)
```

Key discussion:

> **File upload:** "Compress images before uploading. Use multi-part form data with progress indicator. Make uploads resumable in case of network interruption."
> **Regulatory timelines:** "Card networks give 120 days to dispute. The UI must show deadline clearly — 'You have 45 days remaining to dispute this transaction.'"
> **Provisional credit:** "Balance might temporarily increase during investigation, then decrease if denied. The UI must distinguish provisional vs settled amounts."

PART 5: CROSS-CUTTING CONCERNS

Mention these for ANY scenario:

Caching Strategy

```
┌──────────────────────────────────────────────────────────────┐
│ Cache-first strategy for transaction list:                   │
│                                                              │
│ 1. Show cached data immediately (fast UI)                    │
│ 2. Fetch fresh data in background                            │
│ 3. Diff and update only changed items                        │
│ 4. Cache invalidation on push notification                   │
│                                                              │
│ Cache storage: Encrypted CoreData for transactions           │
│ TTL: 5 minutes for list, 30 seconds for rates                │
└──────────────────────────────────────────────────────────────┘
```

Offline Handling

```
┌──────────────────────────────────────────────────────────────┐
│ Offline-first for read operations:                           │
│ • Show cached transactions when offline                      │
│ • Show "Offline" banner                                      │
│ • Disable actions that need network (dispute, split)         │
│ • Queue actions locally, sync when online                    │
│ • Card freeze/unfreeze must show warning if offline          │
└──────────────────────────────────────────────────────────────┘
```

Security

```
┌──────────────────────────────────────────────────────────────┐
│ • Certificate pinning on all API calls                       │
│ • Keychain for tokens (not UserDefaults)                     │
│ • Biometric auth for sensitive actions                       │
│ • Screen protection (blur when backgrounded)                 │
│ • Jailbreak detection                                        │
│ • No sensitive data in logs                                  │
└──────────────────────────────────────────────────────────────┘
```

Feature Flags

```
┌──────────────────────────────────────────────────────────────
```

```
│ • Every new feature behind a flag                    │
│ • Gradual rollout: 5% → 25% → 50% → 100%             │
│ • Monitor per-flag: crash rate, error rate, funnel   │
│ • Kill switch for emergency disable                  │
│ • Country-specific rollout (like BFX at PayPal)      │
└──────────────────────────────────────────────────────┘
```

Analytics / Monitoring

```
┌──────────────────────────────────────────────────────┐
│ • Push notification delivery rate                    │
│ • Notification → app open conversion rate            │
│ • 3DS approval rate and time-to-approve              │
│ • Transaction list load time (p50, p95, p99)         │
│ • API error rate per endpoint                        │
│ • Crash-free rate > 99.9%                            │
│ • Card freeze response time                          │
└──────────────────────────────────────────────────────┘
```

PART 6: THE TALK TRACK — WHAT TO SAY WHEN

Opening (when they present the problem):

> "Let me make sure I understand the customer's journey first. They [restate the user flow]. Is that right?"

Before drawing:

> "I'll start with the high-level flow — external systems, backend, and mobile. Then I'll zoom into the mobile architecture using VIPER, which I know Wise uses."

While drawing high-level:

> "Events flow from [source] through Wise's backend via [Kafka/API] to the mobile app via [push/API]. Let me zoom into the mobile side..."

While drawing VIPER:

> "I'm using VIPER because Wise's codebase uses it and I'm experienced with it. The View is completely dumb, the Presenter coordinates, the Interactor handles business logic, the Router owns navigation, and Entities are pure data."

On data flow:

> "The Interactor calls the Repository, which decides: check cache first, then fetch from network if stale. This gives the user instant UI from cache while fresh data loads in the background."

On trade-offs (ALWAYS state both sides):

> "I'd choose [A] over [B] because [reason]. The trade-off is [what you lose with A], but in this context [why that's acceptable]."

On testing:

> "Each VIPER component is independently testable — mock the View to test Presenter, mock the Repository to test Interactor. For a fintech app processing £26 billion, this isolation is essential."

Connecting to your experience:

> "This is similar to what I built at PayPal with BFX — [specific connection]. The difference is [how Wise's context differs]."

When they add requirements:

> "I'd create a new VIPER module for this rather than bloating the existing one. Wrap it behind a feature flag. The trade-off is [X vs Y]."

Closing:

> "For production, I'd add: certificate pinning, monitoring dashboards for [specific metrics], feature flags for gradual rollout, and accessibility support. These are standard for any fintech mobile feature."

PART 7: DRAWING PRACTICE GUIDE

What to draw (practice on paper 3 times):

Drawing 1: High-Level (2 min)

```
[External] → [Wise Backend] → [Mobile App]

Draw 3 boxes with arrows.
Label: what enters, what exits.
```

Drawing 2: Backend (2 min)

```
[API Gateway] → [Service] → [Event Bus / Kafka]
                    ↓
          [Notification Service] + [Transaction Service]

Draw boxes with event flow arrows.
```

Drawing 3: Mobile VIPER (3 min)

```
[View] ←→ [Presenter] ←→ [Interactor]
          ↕            ↕
       [Router]     [Repository]
                  ↙      ↘
            [APIClient]  [Cache]
```

Draw the 5 VIPER components with connections.
Label each with its ONE responsibility.

Drawing 4: Data flow for any feature (3 min)

```
User taps → View sends action → Presenter → Interactor
                                  ↓
                              Repository
                              ↙      ↘
                        Check cache → Cache hit → Return
                                  Cache miss → API call → Store → Return
```

Practice these 4 drawings on paper until you can draw each in under 2 minutes. Speed matters because you only have ~15 min for the main design.

PART 8: QUESTIONS TO ASK THEM

Pick 2:

1. "How does the mobile team handle server-driven UI for card features? Is Dynamic Forms used in the Spend product too?"
2. "What's the latency target for card transaction notifications — from card tap to push delivery?"
3. "How do you manage the VIPER to TCA migration? Is it screen-by-screen, or is there a broader strategy?"
4. "What's the biggest mobile technical challenge in the Spend product right now?"
5. "How do iOS and Android teams coordinate on feature parity for card features?"

PART 9: CHECKLIST — BEFORE ROUND 3

□ Can I draw high-level architecture in 2 minutes?
□ Can I draw VIPER module in 3 minutes?
□ Can I walk through card transaction flow end-to-end?

- □ Can I explain 3DS push notification flow?
- □ Do I have 3 experience stories ready (2 min each)?
- □ Can I explain WHY VIPER for every component?
- □ Can I handle "now add X" by creating new modules?
- □ Do I mention: testing, feature flags, monitoring, security?
- □ Do I connect answers to BFX/PayPal?
- □ Do I start from the CUSTOMER journey, not the database?
- □ Do I state trade-offs: "I chose A over B because..."?
- □ Do I have 2 questions to ask them?